A aprendizagem de Máquina - II

Atividade para Compensação de Aula (19.09.2026)
Prof. Luis Claudio

Aluno: Michel Cirilo de Oliveira

Situação-Problema 01 — Configuração e treinamento de uma MLP

Uma empresa deseja utilizar uma rede neural para classificar clientes entre duas categorias. Após a preparação dos dados, cada cliente é representado por **20 atributos numéricos**. A equipe construiu inicialmente uma MLP com duas camadas ocultas, contendo **64 e 32 neurônios**, ambas utilizando ReLU, e uma camada de saída com **1 neurônio e Sigmoid**.

O treinamento foi configurado com batch_size = 128, epochs = 40, learning_rate = 0,001 e otimizador Adam. Durante os testes, a equipe percebeu que diferentes configurações produzem resultados distintos e decidiu avaliar alterações tanto na arquitetura quanto no processo de treinamento.

Questões

1) Identifique, entre as configurações apresentadas, quais correspondem a **hiperparâmetros de arquitetura** e quais correspondem a **hiperparâmetros de treinamento**. Explique também por que pesos e biases não pertencem a nenhuma dessas duas categorias.

- ***Hiperparâmetros de Arquitetura:*** *Número de camadas ocultas (2), quantidade de neurônios nessas camadas (64 e 32), número de neurônios na camada de saída (1) e as funções de ativação escolhidas (ReLU para as ocultas, Sigmoid para a saída).*
- ***Hiperparâmetros de Treinamento:*** *Tamanho do lote ou batch_size (128), número de épocas (40), taxa de aprendizado ou learning_rate (0,001) e o tipo de otimizador utilizado (Adam).*
- ***Pesos e Biases:*** *Eles não são hiperparâmetros pois* ***não são configurados manualmente antes do início do treinamento****. Eles são os* ***parâmetros internos do modelo (variáveis treináveis)****, e seus valores são aprendidos, ajustados e descobertos automaticamente pelo algoritmo de otimização através dos dados ao longo do processo de treinamento.*

2) A equipe pretende testar diferentes quantidades de camadas ocultas, neurônios e funções de ativação. Explique por que essas escolhas precisam ser avaliadas experimentalmente e como elas podem afetar a capacidade de representação da rede.

- *Redes muito rasas ou com poucos neurônios podem não conseguir aprender as minúcias dos dados (causando underfitting).*
- *Redes excessivamente profundas ou largas possuem capacidade demais, podendo decorar o ruído dos dados de treino em vez de aprender os padrões gerais (causando overfitting). Já as funções de ativação introduzem a não linearidade no modelo; escolhê-las inadequadamente pode dificultar a convergência e até travar o aprendizado (ex: sofrendo do desvanecimento do gradiente).*

3) Considere agora alterações no batch_size, no número de épocas e no learning_rate. Explique como cada uma dessas alterações pode modificar o processo de treinamento, incluindo quantidade

de atualizações, estabilidade do gradiente, velocidade de aprendizado e possibilidade de underfitting ou overfitting.

- ***Batch_size:*** *Define quantas amostras a rede analisa antes de atualizar os pesos. Lotes muito pequenos geram muitas atualizações rápidas por época e um gradiente mais ruidoso/instável (o que às vezes ajuda o modelo a sair de mínimos locais, mas causa oscilação). Lotes muito grandes produzem gradientes mais estáveis e confiáveis, mas têm poucas atualizações por época e o aprendizado é mais lento e custoso computacionalmente.*
- ***Épocas (Epochs):*** *O número de vezes que a rede "vê" todos os dados. Poucas épocas causam underfitting por interromperem o treinamento antes de o modelo aprender; muitas épocas aumentam muito a probabilidade de overfitting, pois a rede começa a memorizar as peculiaridades das amostras de treino.*
- ***Learning_rate:*** *Dita o tamanho do passo com que os pesos são atualizados. Valores altos podem resultar em instabilidade no gradiente e divergência (o modelo oscila fora do mínimo local). Valores muito pequenos dão extrema estabilidade de treinamento, mas exigem muito mais tempo e épocas, podendo fazer o modelo ficar preso prematuramente num mínimo local.*

4) Explique, de forma integrada, o papel da **função de perda, do gradiente, do learning rate e do otimizador** durante o treinamento. Deixe claro por que o gradiente não é um hiperparâmetro.

*Durante o aprendizado, a rede processa dados e gera um resultado; a **função de perda** calcula a diferença matemática (o erro) entre o resultado previsto e a classificação real. A partir dessa perda e usando o backpropagation, calcula-se o **gradiente**, que indica a direção e qual a inclinação da curva de erro em relação aos pesos (como a bússola guiando em direção ao erro mínimo). O **learning rate** é o multiplicador que decide o quão longo será o passo que tomaremos na direção apontada pelo gradiente. Por fim, o **otimizador** é o maestro matemático (como o Adam) que recolhe esses valores e de fato altera os pesos, muitas vezes acrescentando momentos e suavizações para otimizar os saltos. O gradiente não é um hiperparâmetro porque **ele é uma derivada calculada dinamicamente**: é uma grandeza que muda a cada iteração resultante da equação da função de perda com os pesos atuais, e não um valor fixado de antemão por quem programa a rede.*

Situação-Problema 02 — Classificação de imagens utilizando CNN

Uma indústria deseja classificar imagens coloridas de produtos para identificar possíveis defeitos. Cada imagem possui dimensão **32 × 32 × 3**. A primeira camada convolucional utiliza **16 filtros 3 × 3**, stride 1, padding same e função ReLU. A rede possui posteriormente outra camada convolucional, uma operação de Max Pooling e, antes das camadas densas, uma operação Flatten.

Durante o desenvolvimento, alguns integrantes da equipe confundiram conceitos como canais, filtros, mapas de características e quantidade de parâmetros, além de questionarem por que utilizar uma CNN em vez de simplesmente transformar toda a imagem em um vetor e utilizar uma MLP.

Questões

5) Explique como um filtro 3 × 3 atua sobre uma imagem RGB. Indique a dimensão completa de cada filtro, como os três canais participam do cálculo e quantos mapas de características são produzidos pelos 16 filtros.

> *Um filtro age deslizando sobre toda a superfície da imagem. Como a imagem RGB tem 3 canais de cor, a dimensão completa do filtro deve abraçar a profundidade dos dados. Logo, a **dimensão completa de cada filtro será 3 × 3 × 3** (altura, largura e os 3 canais). Ao se posicionar em um setor de* 3×3 *da imagem, o filtro alinha um peso respectivo para cada pixel do canal Red, outro para os pixels no canal Green e outro no Blue (totalizando* $3 \times 3 \times 3 = 27$ *multiplicações). Os 27 resultados são somados juntos (produto escalar 3D) a um termo de bias gerando apenas um único valor para aquele pedaço na saída. Já que a camada usa **16 filtros diferentes**, serão gerados **16 mapas de características** isolados na saída.*

6) Calcule a quantidade de parâmetros treináveis dessa primeira camada convolucional. Em seguida, explique por que essa quantidade não aumenta quando a altura e a largura da imagem aumentam, desde que sejam mantidos a quantidade e o tamanho dos filtros e a quantidade de canais.

- ***Cálculo:** Tamanho total por filtro = 3 (largura) × 3 (altura) × 3 (canais) = 27 pesos. Adicionando o bias (cada filtro ganha 1): 27 + 1 = 28 parâmetros por filtro. Como temos 16 filtros: 16 × 28 = **448 parâmetros treináveis**.*
- ***Motivo de não aumentar com o tamanho da imagem:** As CNNs utilizam um conceito chamado **compartilhamento de pesos (weight sharing)**. Independentemente se a imagem possui 32x32 pixels ou 4K, exatamente os mesmos filtros contendo os mesmos 448 parâmetros deslizarão e varrerão a imagem inteira. O tamanho dos dados afeta o tamanho do mapa de saída (features map), mas não o número de parâmetros da camada.*

7) Explique como **stride e padding** influenciam as dimensões espaciais dos mapas de características. Em sua resposta, diferencie same e valid.

> *O stride dita o "passo" com que o filtro salta pela imagem; quanto maior o stride (*> 1*), menor espacialmente será o mapa de saída (redução de dimensionalidade), pois o filtro ignora posições intermediárias. O padding é a inserção controlada de bordas falsas (normalmente de zeros) em torno da imagem.*

- ***Padding Valid:*** *Significa que não há nenhum padding (zero bordas artificiais). Como o filtro exige espaço completo para funcionar, as beiradas originais acabam encolhendo na varredura. Assim, a saída será invariavelmente menor que a imagem de entrada.*
- ***Padding Same:*** *Injeta-se padding suficiente de zeros na imagem de entrada de modo a garantir que o mapa de características na saída mantenha perfeitamente* ***a mesma largura e altura*** *originais da imagem de entrada (assumindo que o stride seja* 1*).*

8) Explique o papel de **Max Pooling e Flatten** na arquitetura e, de forma geral, por que uma CNN tende a ser mais adequada que uma camada densa aplicada diretamente aos pixels quando se trabalha com imagens.

- ***Max Pooling:*** *Desempenha o papel de downsampling (redução da dimensão da imagem/mapas de características), pegando apenas os valores máximos em pequenas janelas (ex:* 2×2*). Isso reduz a sobrecarga computacional de parâmetros posteriores, ajuda a descartar o ruído da imagem extraindo a essência (destaque mais ativo da feature) e cria resiliência do modelo a pequenas distorções de movimento (invariância a translações).*
- ***Flatten:*** *Converte e empilha toda a massa de cubos/mapas das imagens processadas (tensores em 3D ou mais) em um único array plano em linha reta (1D). É a ponte mandatória que prepara os dados antes de jogá-los para a camada final das redes clássicas totalmente conectadas (MLP) que dirão qual é a classe do produto.*
- ***Por que a CNN é mais adequada:*** *Aplicar uma camada Densa diretamente a uma imagem rasga todo o formato de "quadro" da imagem e desconsidera as interações locais entre os pixels vizinhos, ignorando linhas, formatos geométricos, texturas, e afins. Além disso, uma simples camada densa para cada pixel geraria explosões colossais no número de parâmetros, dificultando o treino e provocando violento overfitting. A CNN, varrendo por blocos, aproveita e rastreia padrões espaciais por meio da sua localidade natural e seu brilhante uso eficiente de parâmetros compartilhados.*